

Ground Robot Kinematics

Juan Francisco Rascón Crespo

Tuesday 3rd March, 2026

1 Wheel Constraints

Complete

2 Base with Three Active Steerable Wheels (Three Swerve Drive)

A base with three active steerable wheels (three swerve drive) is an omnidirectional locomotion system. It allows a ground robot to move in any direction without first changing the chassis orientation. Each wheel is oriented independently, which gives the robot maneuverability and flexibility.

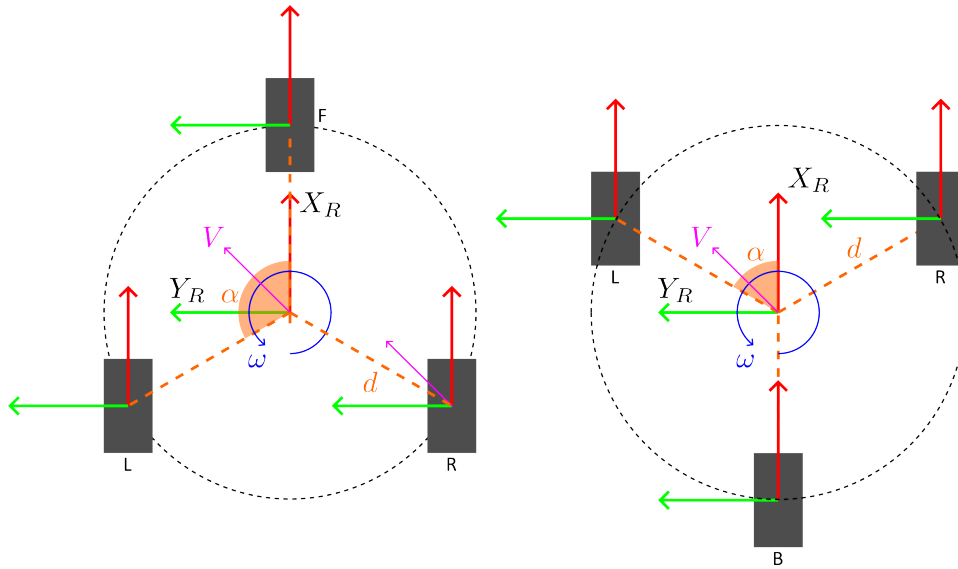


Figure 1: Common wheel configurations in a three swerve drive system (left: inverted-Y configuration, right: Y configuration)

The typical wheel layout is triangular. The two most common configurations are:

- Inverted-Y configuration: One wheel is placed at distance d from the center on the positive X_R axis. The other two are also placed at distance d , at angles α° and $-\alpha^\circ$ with respect to the positive X_R axis, with $\alpha \in [90^\circ, 180^\circ)$.
- Y configuration: One wheel is placed at distance d from the center on the negative X_R axis. The other two are also placed at distance d , at angles α° and $-\alpha^\circ$ with respect to the positive X_R axis, with $\alpha \in (0^\circ, 90^\circ]$.

Figure 1 shows both configurations.

The following sections present inverse and forward kinematics for a robot with three active wheels under the previous configurations. A more general formulation can also be considered, with different distances to the center or wheel angles that are not symmetric. However, those configurations are less common and, in this context, only complicate the expressions without practical advantages, since in industry the previously described configurations are predominantly used.

For the inverted-Y configuration, $i \in \{F, L, R\}$ is used to identify each wheel. For the Y configuration, $i \in \{B, L, R\}$ is used.

The base linear velocity is denoted as:

$$\mathbf{V} = \begin{bmatrix} V_x \\ V_y \\ 0 \end{bmatrix} \quad (1)$$

and is expressed in m/s.

The base angular velocity is denoted as:

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega_z \end{bmatrix} \quad (2)$$

The linear velocity of each wheel i is expressed as:

$$\begin{aligned} \mathbf{V}_i &= \mathbf{V} + \boldsymbol{\omega} \times \mathbf{d}_i \\ &= \mathbf{V} + [\boldsymbol{\omega}]_{\times} \mathbf{d}_i \end{aligned} \quad (3)$$

where $\mathbf{d}_i$ is the position vector of wheel i with respect to the base center.

The cross product between $\boldsymbol{\omega}$ and $\mathbf{d}_i$ can be written as a matrix product through the skew-symmetric operator $[\cdot]_{\times}$: $\boldsymbol{\omega} \times \mathbf{d}_i = [\boldsymbol{\omega}]_{\times} \mathbf{d}_i$, and $[\boldsymbol{\omega}]_{\times}$ is

defined as:

$$[\boldsymbol{\omega}]_{\times} \triangleq \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}$$

Therefore, considering $\boldsymbol{\omega} = \begin{bmatrix} 0 & 0 & \omega_z \end{bmatrix}^T$ and $\mathbf{d}_i = \begin{bmatrix} d \cos \alpha_i & d \sin \alpha_i & 0 \end{bmatrix}^T$:

$$\mathbf{V}_i = \mathbf{V} + \begin{bmatrix} -\omega_z d \sin \alpha_i \\ \omega_z d \cos \alpha_i \\ 0 \end{bmatrix} = \begin{bmatrix} V_x - \omega_z d \sin \alpha_i \\ V_y + \omega_z d \cos \alpha_i \\ 0 \end{bmatrix} \quad (4)$$

The magnitude of the linear velocity of each wheel i is given by:

$$\|\mathbf{V}_i\| = \sqrt{(V_x - \omega_z d \sin \alpha_i)^2 + (V_y + \omega_z d \cos \alpha_i)^2} = \omega_i r \quad (5)$$

where r is the wheel radius and ω_i is the angular speed of wheel i .

Obviously, all wheels have the same radius r .

2.1 Inverse Kinematics

Inverse kinematics is the process of computing wheel angular speeds ω_i and steering angles θ_i from the base linear velocity $\mathbf{V}$ and base angular velocity $\boldsymbol{\omega}$.

The orientation of each wheel i , θ_i , is obtained from its linear velocity $\mathbf{V}_i$:

$$\theta_i = \arctan 2(V_{iy}, V_{ix}) = \arctan 2(V_y + \omega_z d \cos \alpha_i, V_x - \omega_z d \sin \alpha_i) \quad (6)$$

Then, the angular speed of each wheel i , ω_i , is obtained from its linear velocity $\mathbf{V}_i$ and radius r :

$$\omega_i = \frac{\|\mathbf{V}_i\|}{r} = \frac{\sqrt{(V_x - \omega_z d \sin \alpha_i)^2 + (V_y + \omega_z d \cos \alpha_i)^2}}{r} \quad (7)$$

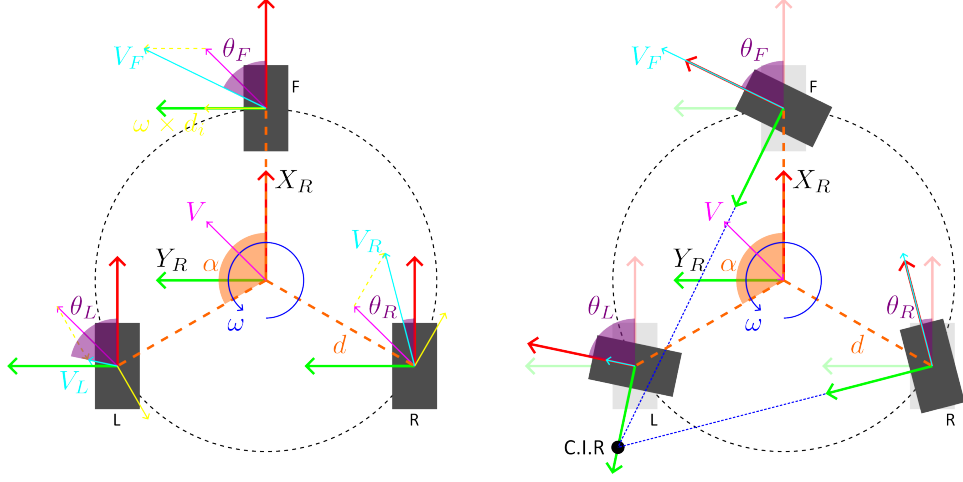


Figure 2: Inverse kinematics of a robot with inverted-Y configuration

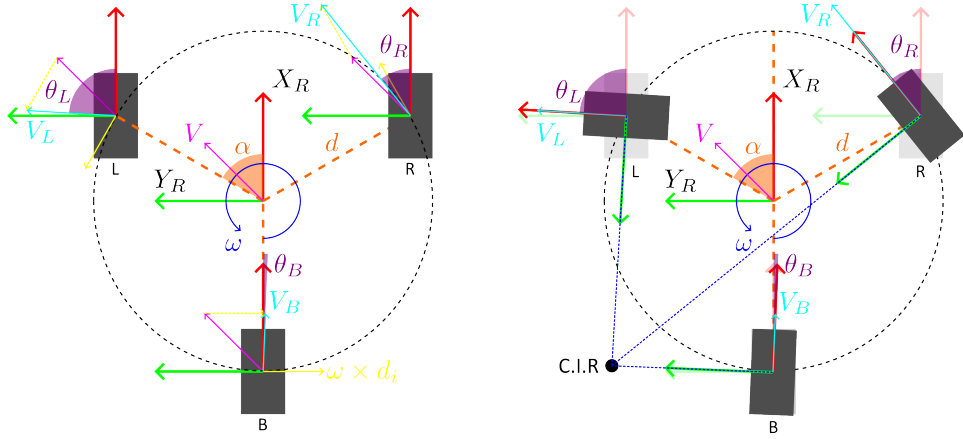


Figure 3: Inverse kinematics of a robot with Y configuration

2.2 Forward Kinematics

Forward kinematics is the process of computing the base linear velocity $\mathbf{V}$ and base angular velocity ω from wheel angular speeds ω_i and wheel orientations θ_i .

There are 6 input variables: wheel angular speeds ω_i and orientations θ_i for each wheel i . The goal is to compute 3 output variables: base linear velocity $\mathbf{V} = (V_x, V_y, 0)^T$ and base angular velocity $\omega = (0, 0, \omega_z)^T$. Therefore, the system is overdetermined and is robustly solved using least squares. For the inverted-Y configuration, $i \in \{F, L, R\}$ is used (Front, Left, Right). For the Y configuration, this becomes $i \in \{B, L, R\}$ (Back, Left, Right).

For each wheel i , define the rolling unit vector:

$$\mathbf{t}_i = \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \\ 0 \end{bmatrix} \quad (8)$$

The linear velocity of the wheel along its rolling direction is:

$$\mathbf{V}_i = r\omega_i \mathbf{t}_i = r\omega_i \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \\ 0 \end{bmatrix} \quad (9)$$

By equating (4) and (9), for each wheel i :

$$\begin{aligned} V_x - \omega_z d \sin \alpha_i &= r\omega_i \cos \theta_i \\ V_y + \omega_z d \cos \alpha_i &= r\omega_i \sin \theta_i \end{aligned} \quad (10)$$

In the ideal case (no noise and no slip), any subset of 3 independent equations allows solving $\mathbf{V}$ and ω exactly. From an ideal mathematical point of view, the procedure is to build a linear system and solve it with least squares.

$$A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} = \begin{bmatrix} V_x \\ V_y \\ \omega_z \end{bmatrix} \quad (11)$$

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|A\mathbf{x} - \mathbf{b}\|_2^2 = (A^\top A)^{-1} A^\top \mathbf{b} \quad (12)$$

As reference, for the inverted-Y configuration ($i \in \{F, L, R\}$) the following matrices A and $\mathbf{b}$ are used:

$$A_{\text{inverted-Y}} = \begin{bmatrix} 1 & 0 & -d \sin \alpha_F \\ 0 & 1 & d \cos \alpha_F \\ 1 & 0 & -d \sin \alpha_L \\ 0 & 1 & d \cos \alpha_L \\ 1 & 0 & -d \sin \alpha_R \\ 0 & 1 & d \cos \alpha_R \end{bmatrix} \quad (13)$$

$$\mathbf{b}_{\text{inverted-Y}} = r \begin{bmatrix} \omega_F \cos \theta_F \\ \omega_F \sin \theta_F \\ \omega_L \cos \theta_L \\ \omega_L \sin \theta_L \\ \omega_R \cos \theta_R \\ \omega_R \sin \theta_R \end{bmatrix} \quad (14)$$

with $\alpha_F = 0^\circ$, $\alpha_L = \alpha$, and $\alpha_R = -\alpha$.

For the Y configuration ($i \in \{B, L, R\}$) the following matrices A and $\mathbf{b}$ are used:

$$A_Y = \begin{bmatrix} 1 & 0 & -d \sin \alpha_B \\ 0 & 1 & d \cos \alpha_B \\ 1 & 0 & -d \sin \alpha_L \\ 0 & 1 & d \cos \alpha_L \\ 1 & 0 & -d \sin \alpha_R \\ 0 & 1 & d \cos \alpha_R \end{bmatrix} \quad (15)$$

$$\mathbf{b}_Y = r \begin{bmatrix} \omega_B \cos \theta_B \\ \omega_B \sin \theta_B \\ \omega_L \cos \theta_L \\ \omega_L \sin \theta_L \\ \omega_R \cos \theta_R \\ \omega_R \sin \theta_R \end{bmatrix} \quad (16)$$

with $\alpha_B = 180^\circ$, $\alpha_L = \alpha$, and $\alpha_R = -\alpha$.

Note that the least-squares solution in (12) is valid whenever $A^\top A$ is invertible.

In real conditions, the 6 equations do not match exactly due to several causes:

- sensor noise
- wheel slip
- geometry or calibration errors
- encoder quantization

Although the closed-form expression (12) is useful for analysis, in numerical implementation it is recommended to avoid explicit computation of $(A^\top A)^{-1}$, since for the causes above A can be ill-conditioned or even singular, which can make $A^\top A$ non-invertible or its inverse numerically unstable.

From an implementation point of view, the recommended approach is to solve least squares with numerically stable methods, instead of using explicit inversion of $(A^\top A)$.

Two approaches can be followed:

- Solve the least-squares problem directly via QR decomposition (orthogonal-triangular decomposition).
- Solve it via the Moore-Penrose pseudoinverse:

$$\hat{\mathbf{x}} = A^\dagger \mathbf{b}$$

In this approach, the usual way to compute $A^\dagger$ robustly is singular value decomposition (SVD).

As a practical implementation criterion:

- **QR (orthogonal-triangular decomposition):**
 - Advantages: lower computational cost and higher speed.
 - Drawback: lower robustness when the system is highly ill-conditioned or nearly singular.
 - Recommended use: online estimation of $\hat{\mathbf{x}}$ when the problem is reasonably well-conditioned.
- **SVD (singular value decomposition):**
 - Advantages: higher numerical robustness and better handling of ill-conditioned or nearly singular systems; it also allows diagnosis of the effective rank of the system.
 - Drawback: higher computational cost.
 - Recommended use: when numerical stability and robustness against noise or poor conditioning are prioritized.

To choose between QR and SVD in this problem, the following procedure can be used:

- Build A from geometry (d and α_i).
- Evaluate its conditioning once (for example, at startup), since A is constant if the geometry does not change.
- Compute the condition number of A :

$$\kappa(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}$$

where $\sigma_{\max}(A)$ and $\sigma_{\min}(A)$ are, respectively, the largest and smallest singular values of A .

- To obtain singular values, use singular value decomposition:

$$A = U\Sigma V^\top$$

where U and V are orthogonal matrices, and Σ is a diagonal (or rectangular-diagonal) matrix whose diagonal elements are the singular values σ_i .

- Equivalent practical form:

* Compute $A^\top A$.

- * Obtain its eigenvalues λ_i .
- * Compute $\sigma_i = \sqrt{\lambda_i}$.
- * Compute explicitly:

$$\sigma_{\max} = \sqrt{\lambda_{\max}(A^T A)}, \quad \sigma_{\min} = \sqrt{\lambda_{\min}(A^T A)}$$

- Choose the method using guideline thresholds for κ :
 - $\kappa < 10^3$: use QR
 - If $\kappa \geq 10^3$, use SVD
- In code, it is useful to compute the residual as a quality check of the solution:
 - Useful for:
 - * Detecting slip or anomalous readings.
 - * Detecting when the model does not explain the data well at that instant.
 - * Raising flags, diagnostics, or logs.
 - Typical metric:

$$r = \|A\hat{\mathbf{x}} - \mathbf{b}\|_2, \quad r_{\text{rel}} = \frac{\|A\hat{\mathbf{x}} - \mathbf{b}\|_2}{\max(\|\mathbf{b}\|_2, \varepsilon)}$$

In the code example, $\varepsilon = 10^{-12}$ is used to avoid division by zero when $\|\mathbf{b}\|_2$ is very small.

- Practical interpretation:
 - * $r_{\text{rel}} \leq 10^{-3}$: consistent solution.
 - * $10^{-3} < r_{\text{rel}} \leq 10^{-2}$: intermediate quality, monitoring is recommended.
 - * $r_{\text{rel}} > 10^{-2}$: possible slip, high noise, outliers, or miscalibration.
- In online execution, only update $\mathbf{b}$ with wheel measurements and solve with the selected method.
- Re-evaluate only if geometry or calibration changes.

As a complement, the following C++/Eigen example shows a direct implementation of the procedure above:

Listing 1: C++/Eigen recipe to choose QR or SVD and validate the result

```

using Eigen::ComputeThinU;
using Eigen::ComputeThinV;
using Eigen::JacobiSVD;
using Eigen::Matrix;
using Eigen::Matrix3d;
using Eigen::SelfAdjointEigenSolver;
using Eigen::Vector3d;

// A is built once at program startup
Matrix<double, 6, 3> A = build_A(d, alpha);
// kappa(A) = sig_max / sig_min.
Matrix3d AtA = A.transpose() * A;
SelfAdjointEigenSolver<Matrix3d> es(AtA);
// Ascending order
Vector3d lambda = es.eigenvalues();
double sig_min = std::sqrt(std::max(lambda(0), 0.0));
double sig_max = std::sqrt(std::max(lambda(2), 0.0));
double kappa = sig_max / std::max(sig_min, 1e-12);

...
while(data_available) {
    // In each iteration, only b changes.
    // w: wheel angular speed (3x1)
    // theta: wheel orientation (3x1)
    // r: wheel radius
    Matrix<double, 6, 1> b = build_B(w, theta, r);
    Vector3d x_hat;

    if (kappa < 1e3)
    {
        // QR with pivoting
        x_hat = A.colPivHouseholderQr().solve(b);
    }
    else
    {
        auto opts = ComputeThinU | ComputeThinV;
        JacobiSVD<Matrix<double, 6, 3>> svd(A, opts);
        x_hat = svd.solve(b);
    }

    // Quality check (relative residual).
    double res = (A * x_hat - b).norm();

```

```

double res_rel = res / std::max(b.norm(), 1e-12);
constexpr double res_rel_threshold = 1e-2;
if (res_rel > res_rel_threshold)
{
    // warning: low estimate quality
    log_warning("High residual", res_rel);
}

```

A Moore-Penrose Pseudoinverse

This content was removed from the main body and preserved in this appendix as conceptual reference. Although it has high theoretical value, integrating it directly into the main narrative could introduce confusion in the reading flow.

The Moore-Penrose pseudoinverse, $A^\dagger$, generalizes the least-squares expression:

- $A^\dagger$ is defined for any matrix A , even if A is not square or not invertible.
- The least-squares formulation using $A^\dagger$ improves robustness when equations are nearly dependent or affected by noise:

$$\hat{\mathbf{x}} = A^\dagger \mathbf{b}$$

- When A has full column rank (the columns of A are linearly independent), matrix $A^\top A$ is invertible and the least-squares solution can be computed through the closed form $(A^\top A)^{-1} A^\top \mathbf{b}$. In that case, the following equivalence holds:

$$A^\dagger = (A^\top A)^{-1} A^\top$$

From a numerical point of view, singular value decomposition (SVD) is used to compute $A^\dagger$ robustly.